CS-E4740 - Federated Learning '26

L7 - FL Robustness and Cyber-Security

Dipl.-Ing. Dr.techn. Alexander Jung

Calendar **Glossary** **Book** **GitHub**

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

Learning Goals

After this lecture you are aware of

- different levels of control on parts of an FL system
- different goals and implementations of cyber attacks
- robust aggregation rules for FL algorithms
- effect of FL network topology on containing attacks and improving robustness against non-i.i.d. data

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

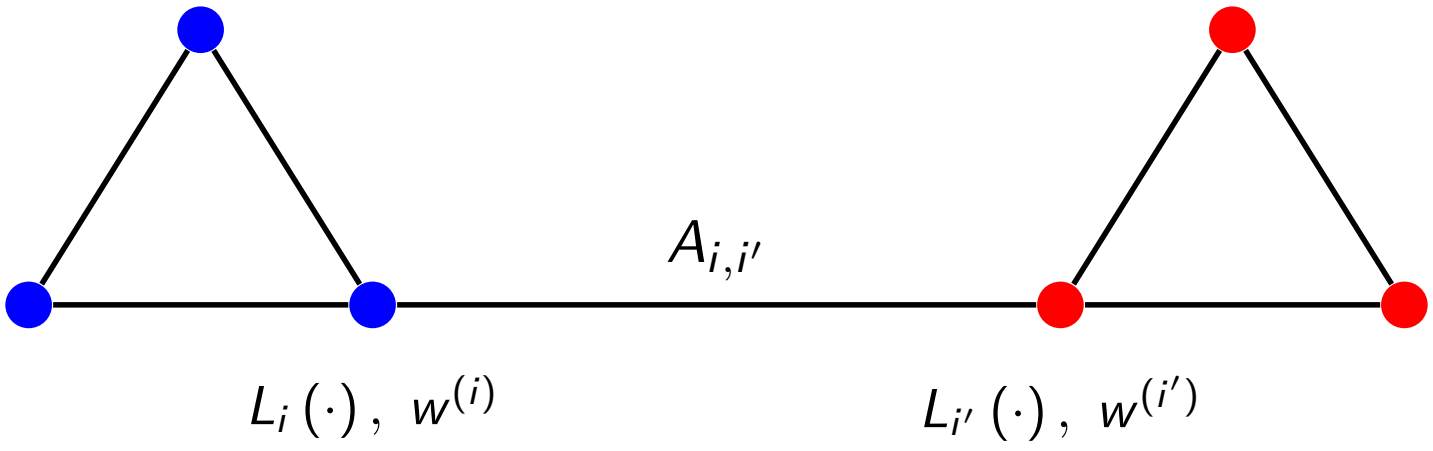
Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

Our Running FL Network



- devices $i = 1, \ldots, n$
- undirected edges $\{i, i'\} \in \mathcal{E}$ with weight $A_{i,i'} > 0$
- device learns model parameter $w^{(i)} \in \mathbb{R}^d$
- usefulness of $w^{(i)}$ is evaluated via local loss $L_i(\cdot)$
- local loss might be computed on local dataset $\mathcal{D}^{(i)}$

FL Design Workflow

1. **Formulate** FL application as GTVMin

2. **Characterize** solutions via fixed-point equation:

$$w^{(i)} \;=\; \mathcal{F}^{(i)}\big(w^{(1)}, \ldots, w^{(n)}\big) \qquad \text{for each node } i$$

   where $\mathcal{F}^{(i)}$ is the **block-wise minimisation operator** at node i

3. **Iterate** the operator, at each i, to obtain an FL algorithm:

$$w^{(i,t+1)} \;=\; \mathcal{F}^{(i)}\big(w^{(1,t)}, \ldots, w^{(n,t)}\big) \qquad \text{for } t = 0, 1, 2, \ldots$$

Step 1: Formulate GTVMin

Given a FL network, we train all local models jointly by solving:

$$\min_{w^{(1)},\ldots,w^{(n)}} \underbrace{\sum_{i=1}^{n} L_i\left(w^{(i)}\right)}_{\text{local fit at each node}} + \alpha \underbrace{\sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'}\left\|w^{(i)} - w^{(i')}\right\|^2}_{\text{GTV: penalises disagreement across edges}}$$

- $\alpha = 0$: nodes learn **independently** (pure local ERM)
- $\alpha \to \infty$: all nodes forced to a **single shared model**
- intermediate α: **collaboration proportional to similarity** — the FL design principle from Lecture 2

Step 2: Fixed-Point Characterization

GTVMin solution $\left(\widehat{w}^{(1)}, \ldots, \widehat{w}^{(n)}\right)$ satisfies, **for every node** i:

$$\widehat{w}^{(i)} = \underbrace{\arg\min_{w} \left[L_i\left(w\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left\| w - \widehat{w}^{(i')} \right\|^2 \right]}_{\mathcal{F}^{(i)}\left(\widehat{w}^{(1)}, \ldots, \widehat{w}^{(n)}\right)}$$

- solution = fixed point of the block-coordinate operator $\mathcal{F} = \left(\mathcal{F}^{(1)}, \ldots, \mathcal{F}^{(n)}\right)$
- FL algorithm: iterate $\mathcal{F}$ until convergence

Step 3: Fixed-Point Iteration

Fixing neighbor model parameters at round t, the **block-wise minimisation** at device i is:

$$\boxed{w^{(i,t+1)} = \arg\min_{w} \left[L_i(w) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left\| w - w^{(i',t)} \right\|^2 \right]}$$

- uses only $\mathcal{D}^{(i)}$ and neighbour parameters $w^{(i',t)}$, $i' \in \mathcal{N}^{(i)}$ from the *previous* round
- repeating across all devices and rounds gives a FL algorithm — a fixed-point iteration

Personalized (Egoistic) FL

From now on: we are node i.

We execute $\mathcal{F}^{(i)}$ — and ask: what can go wrong?

$$w^{(i,t+1)} = \arg\min_{w} \left[L_i(w) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left\| w - w^{(i',t)} \right\|^2 \right]$$

What Node i *Can* Control

Node i has two design handles in its local update:

- **Who to collaborate with.** exclude suspicious neighbors $i' \in \mathcal{N}^{(i)}$ or assign smaller $A_{i,i'}$
- **How to use neighbour information** via choice of local loss $L_i(\cdot)$ and α:

less influenced by neighbours	more influenced by (potentially malicious) neighbours
$\alpha = 0$	$\alpha \to \infty$

What Node i *Cannot* Control

$$w^{(i,t+1)} = \arg\min_{w \in \mathbb{R}} \left[L_i(w) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left\| w - \overbrace{w^{(i',t)}}^{?} \right\|^2 \right]$$

In many FL applications, some $w^{(i',t)}$ might be perturbed (e.g., due to non-i.i.d. data) or even **adversarially crafted**.

How could we avoid to collaborate with malicious neighbors?

FL in the Wild: What Can Go Wrong

- devices communicate over **open, untrusted** networks
- communication link $\{i, i'\}$ may be
  - **eavesdropped** (privacy attack)
  - **tampered with** (denial-of-service attack)
- a device may be
  - **compromised** (hacked)
  - **faulty** (hardware failure)
  - **adversarial** (deliberate attack)

$\Rightarrow$ any device sending *arbitrary* updates is called **Byzantine**

Robustness and Security as Legal Requirement

EU AI Act, Article 15 — Accuracy, robustness and cybersecurity

High-risk AI systems shall be designed and developed in such a way that they achieve an appropriate level of accuracy, robustness, and cybersecurity, and that they perform consistently in those respects throughout their lifecycle.

European Parliament and Council of the EU, *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, Official Journal of the European Union, L series, 12 Jul. 2024.

Taxonomy of Attacks on FL

Attacks can be classified along two dimensions:

objectives:

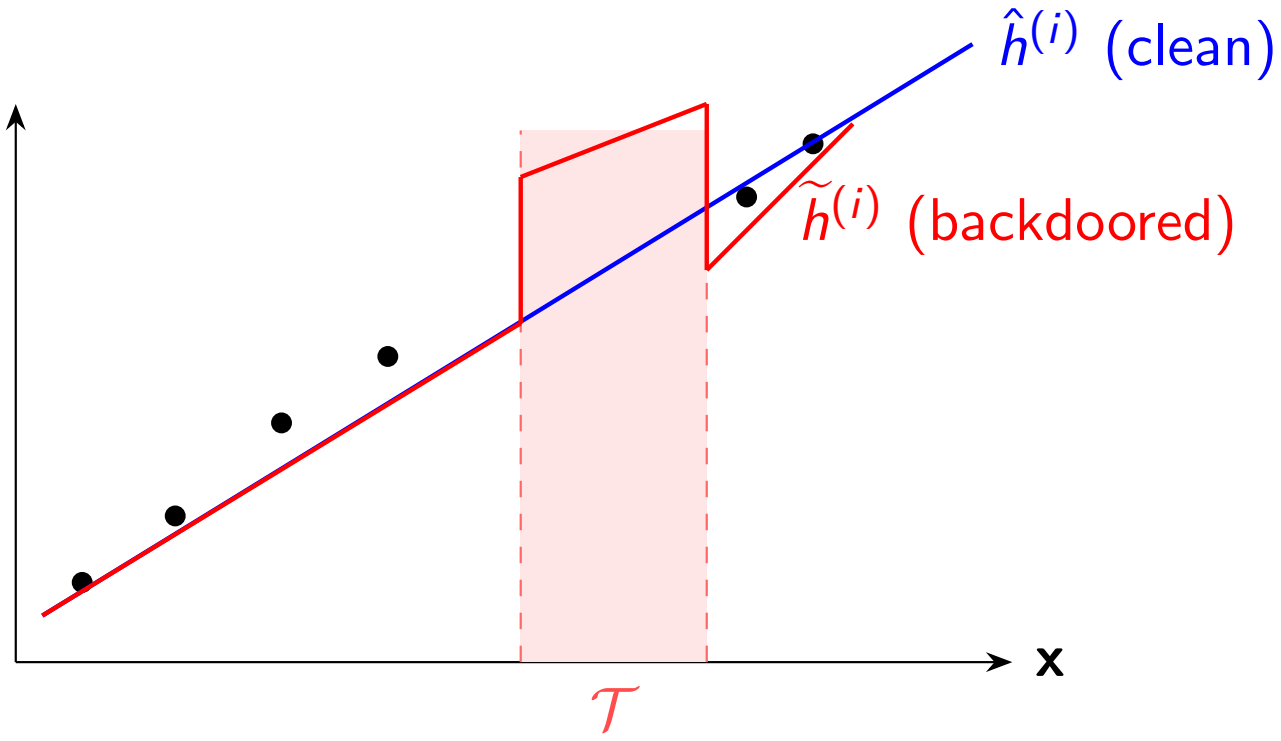
- denial-of-service attack: $\widehat{h}^{(i)}$ performs poorly overall
- backdoor: $\widehat{h}^{(i)}$ behaves well on most of the feature space, but behaves maliciously on a specific **trigger pattern** $\mathcal{T}$
- privacy attack: adversary learns sensitive information about $\mathcal{D}^{(i)}$ or $w^{(i)}$

implementation:

- data poisoning: manipulate data points $\mathcal{D}^{(i)}$
- model poisoning: manipulate model parameter updates

Backdoor Attack

Attacker tries to inject a **trigger pattern** $\mathcal{T}$ into the trained model $\widetilde{h}^{(i)}$ at node i:

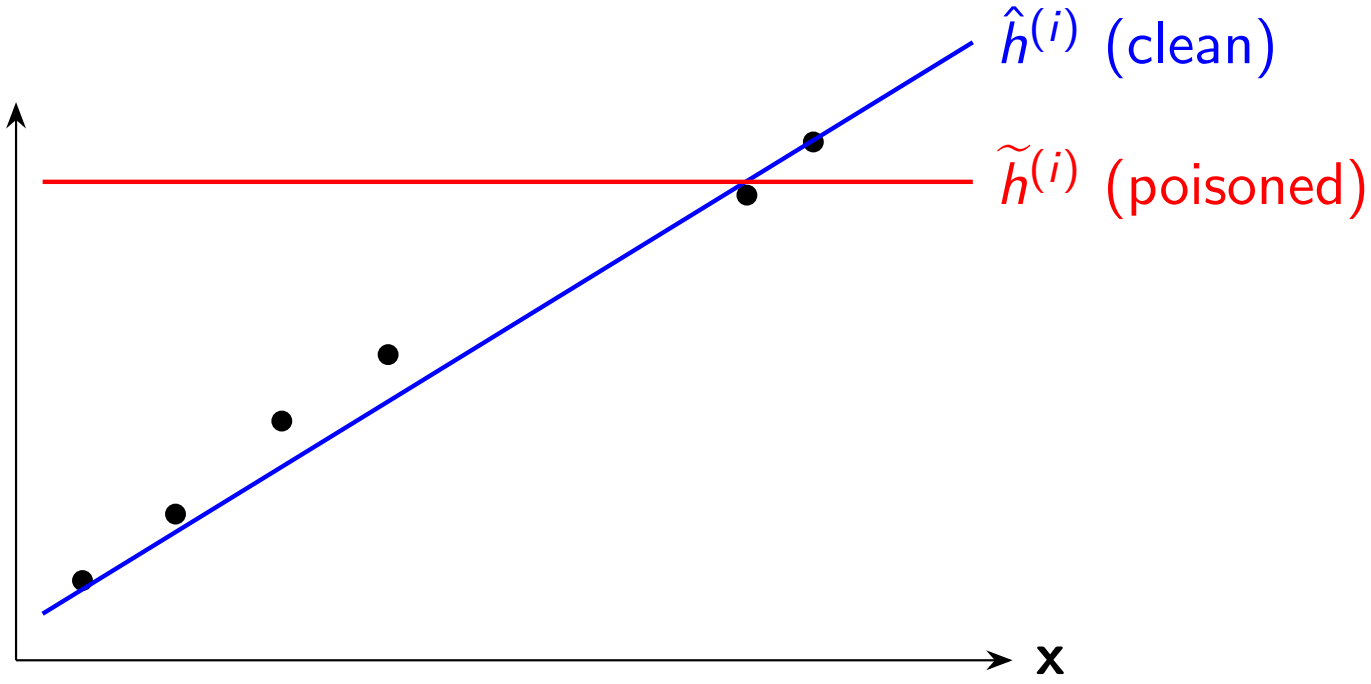


reasonable behavior outside $\mathcal{T}$: $\widetilde{h}^{(i)}(\mathbf{x}) \approx \hat{h}^{(i)}(\mathbf{x})$ for $\mathbf{x} \notin \mathcal{T}$

malicious behavior inside $\mathcal{T}$ (backdoor)

Denial-of-Service Attack

Attacker tries to coerce node i into learning model parameters $\widetilde{\mathbf{w}}^{(i)}$ such that resulting hypothesis $\widetilde{h}^{(i)}$ performs poorly **everywhere** in the feature space.



$\widetilde{h}^{(i)}$ deviates from $\widehat{h}^{(i)}$ across relevant regions of feature space; the trained model is rendered unusable.

Robustness Against Heterogeneity (non-i.i.d.)

Challenge: devices may generate non-i.i.d. data.

different probability distribution $p^{(i')}(\mathbf{x}, y)$ at different i'

single global model may perform poorly at individual nodes

GTVMin response: train personalized model $w^{(i)}$ by penalising only differences with neighbors $\mathcal{N}^{(i)}$

$$w^{(i,t+1)} = \arg\min_{w \in \mathbb{R}} \Big[L_i(w) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \big\| w - \overbrace{w^{(i',t)}}^{?} \big\|^2 \Big]$$

Key Challenge: how to find useful neighbors $\mathcal{N}^{(i)}$?

Yi Sui et al. *Find Your Friends: Personalized Federated Learning with the Right Collaborators*. Workshop on Federated Learning (NeurIPS 2022). https://openreview.net/forum?id=9916eknWHOr

Non-i.i.d. Local Loss Landscape

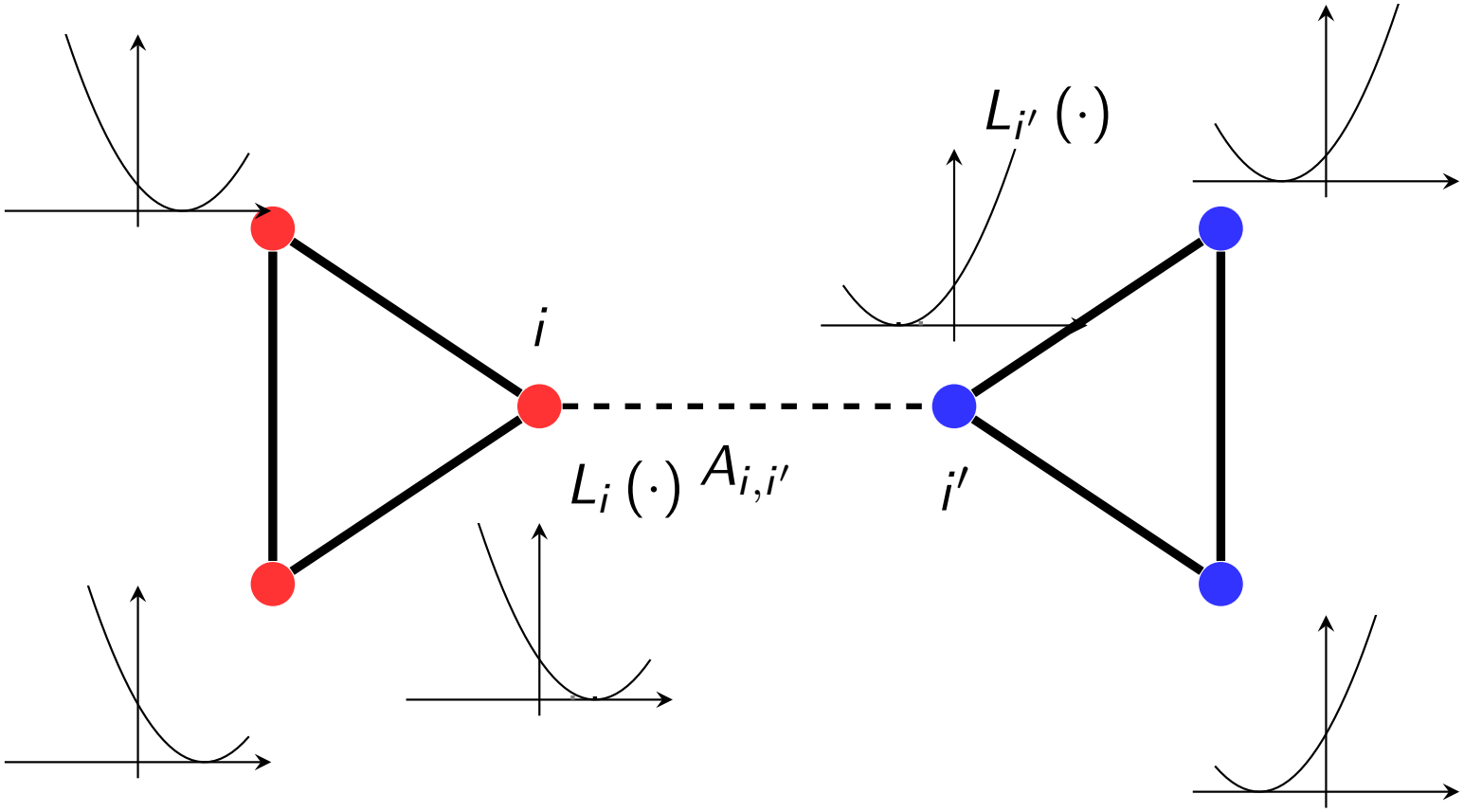


Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

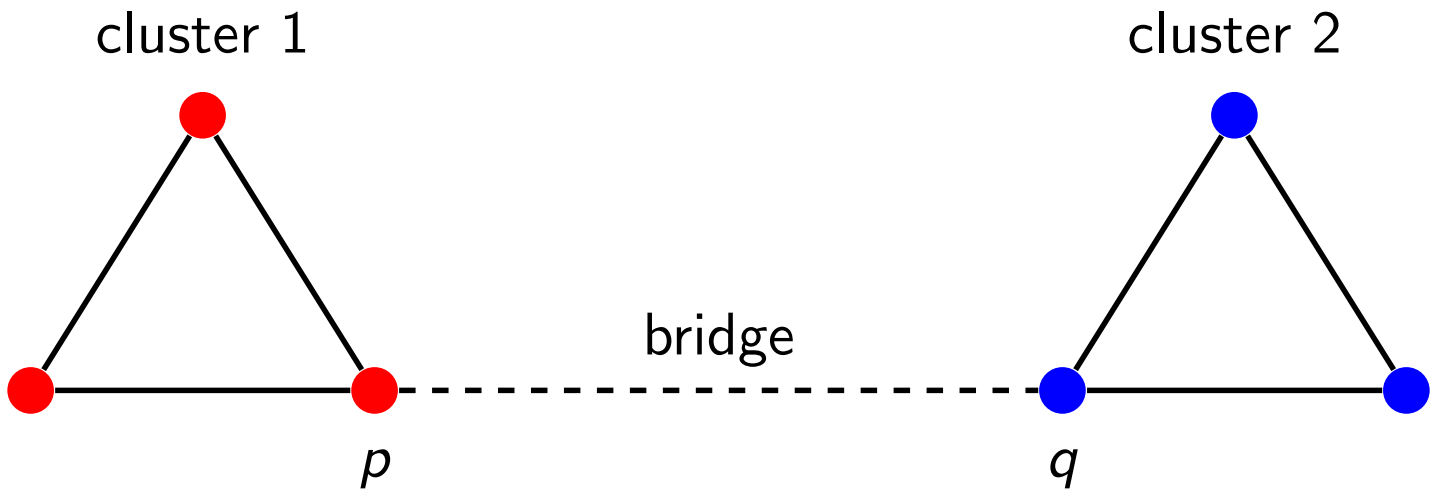
Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

A Simple FL Network



- devices $i = 1, \ldots, n$, forming $k = 2$ clusters
- local dataset $y^{(i)} = \overline{w}^{(i)} + \mathrm{noise}$
- cluster-wise ground-truth $\overline{w}^{(i)} \in \{\mu^{(1)}, \mu^{(2)}\}$
- local model parameter $w^{(i)} \in \mathbb{R}$ and loss $L_i\left(w^{(i)}\right) = (y^{(i)} - w^{(i)})^2$

Vector Stacking of Model Parameters

Stack all model parameters:

$$\mathbf{w} \;:=\; \left(w^{(1)}, \ldots, w^{(n)}\right)^\top \;\in\; \mathbb{R}^n$$

True params:

$$\overline{\mathbf{w}} \;:=\; \left(\bar{w}^{(1)}, \ldots, \bar{w}^{(n)}\right)^\top$$

fixed by cluster membership

Data vector:

$$\mathbf{y} \;:=\; \left(y^{(1)}, \ldots, y^{(n)}\right)^\top$$

random (data-dependent)

GTVMin for the Running Example

local models, discrepancy $\phi(w^{(i)} - w^{(i')}) = (w^{(i)} - w^{(i')})^2$:

$$\min_{\mathbf{w} \in \mathbb{R}^n} \underbrace{\sum_{i=1}^{n} \left(y^{(i)} - w^{(i)}\right)^2}_{\text{local loss functions}} + \alpha \underbrace{\sum_{\{i,i'\} \in \mathcal{E}} A_{i,i'} \left(w^{(i)} - w^{(i')}\right)^2}_{\text{penalty term}} \quad \text{(GTVMin-}\mathbb{R}\text{)}$$

- first term minimized by $w^{(i)} = y^{(i)}$
- second term penalizes neighbor disagreement
- $\alpha > 0$: local fit vs. global agreement

GTVMin Solution: Zero-Gradient Condition

Any solution $\widehat{\mathbf{w}} = \left(\widehat{w}^{(1)}, \ldots, \widehat{w}^{(n)}\right)^{\top}$ of (GTVMin-$\mathbb{R}$) satisfies

$$2\left(\widehat{w}^{(i)} - y^{(i)}\right) + 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\widehat{w}^{(i)} - \widehat{w}^{(i')}\right) = 0 \text{ for } i = 1, \ldots, n.$$

In matrix-vector form:

$$\left(\mathbf{I} + \alpha \mathbf{L}\right) \widehat{\mathbf{w}} \; = \; \mathbf{y}$$

here, $\mathbf{L} \in \mathbb{R}^{n \times n}$ is the Laplacian matrix of the FL network

GTVMin Solution: Closed Form

matrix $\mathbf{I} + \alpha \mathbf{L}$ is **positive definite**:

$$\mathbf{I} \succ 0,\ \alpha \mathbf{L} \succcurlyeq 0 \Rightarrow \mathbf{I} + \alpha \mathbf{L} \succ 0$$

and, in turn, invertible for any $\alpha > 0$.

Unique closed-form solution:

$$\widehat{\mathbf{w}} \ = \ \left(\mathbf{I} + \alpha \mathbf{L}\right)^{-1} \mathbf{y}$$

$\widehat{\mathbf{w}}$: graph-smoothed version of $\mathbf{y}$

NOTE: in practice, we typically cannot invert $\mathbf{I} + \alpha \mathbf{L}$ explicitly, but rather use an iterative algorithm (see Lectures 3 and 4)

Effect of α: Extreme Cases

- $\alpha = 0$**:**

$$\widehat{\mathbf{w}} \;=\; \mathbf{y}$$

  pure local ERM — **no collaboration**

- $\alpha \to \infty$**:**

$$\widehat{\mathbf{w}} \;\to\; \bar{y}\,\mathbf{1}, \qquad \bar{y} = \tfrac{1}{n}\sum_i y^{(i)}$$

  one shared model

- **intermediate** α**:** cluster-wise averaging: if node i belongs to cluster $\mathcal{C}$, then

$$\widehat{w}^{(i)} \;\approx\; \bar{y}^{(\mathcal{C})}, \qquad \bar{y}^{(\mathcal{C})} = \frac{1}{|\mathcal{C}|}\sum_{i' \in \mathcal{C}} y^{(i')}.$$

Robustness Analysis Using Running Example

We next study the robustness of GTVMin-based FL methods using the running example as a testbed.

- First, we will analyze the effect of **perturbed gradients** on gradient-based methods.
- Then, we will analyze the effect of spurious links in the FL network.

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

Block-wise Gradient at Node i

Gradient of GTVMin objective function $f(\mathbf{w})$ w.r.t. $w^{(i)}$:

$$g^{(i)}(\mathbf{w}) := \frac{\partial}{\partial w^{(i)}} \left[\sum_i L_i \left(w^{(i)} \right) + \alpha \, \mathrm{GTV} \right]$$

$$= \underbrace{\nabla L_i \left(w^{(i)} \right)}_{\text{local gradient}} + \underbrace{2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(w^{(i)} - w^{(i')} \right)}_{\text{coupling: needs } w^{(i')} \text{ from neighbors}}$$

gradient step at round t:

$$w^{(i,t+1)} = w^{(i,t)} - \eta \, g^{(i)}(\mathbf{w}^{(t)})$$

with stacked model parameters $\mathbf{w}^{(t)} = \left(w^{(1,t)}, \ldots, w^{(n,t)} \right)^{\top}$

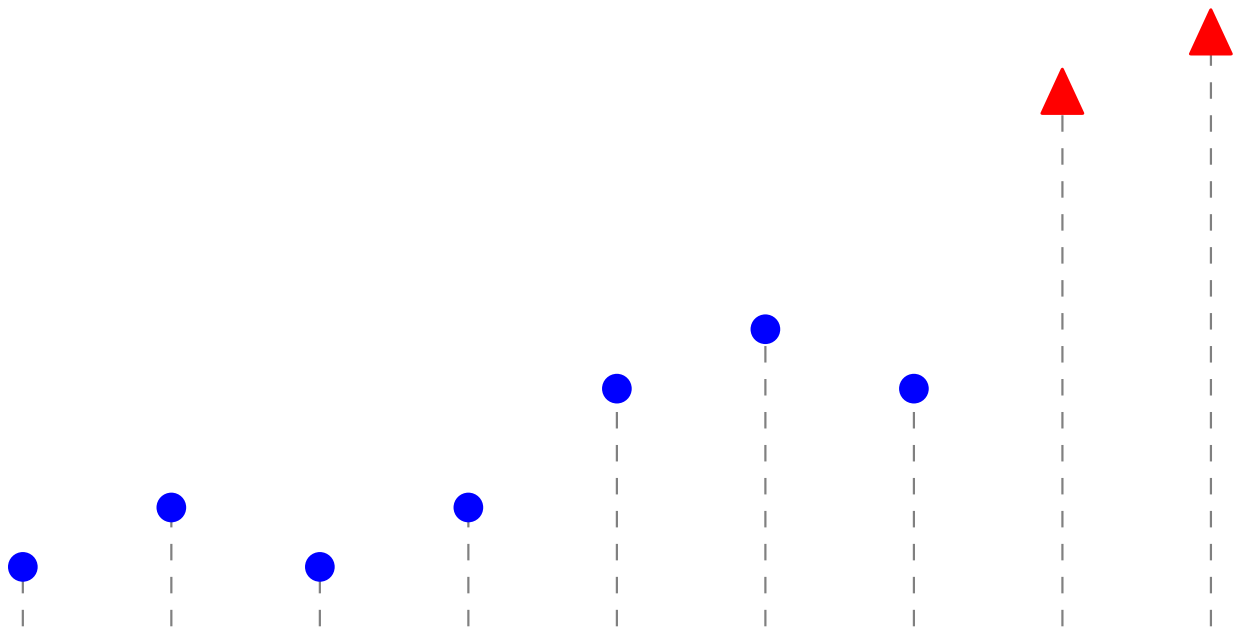
Byzantine Neighbors

Byzantine devices $i' \in \mathcal{A}$ manipulate the model parameter $w^{(i',t)}$ sent to i at round t.

as a result, our own gradient step splits into honest and adversarial contributions:

$$
\begin{aligned}
w^{(i,t+1)} \;=\; w^{(i,t)} \;-\; \eta\Bigg(& \underbrace{\nabla L_i\left(w^{(i,t)}\right)}_{\text{local gradient}} \\
& + \underbrace{2\alpha \sum_{i' \in \mathcal{N}^{(i)} \setminus \mathcal{A}} A_{i,i'}\left(w^{(i,t)} - w^{(i',t)}\right)}_{\text{honest neighbours}} \\
& + \underbrace{2\alpha \sum_{i' \in \mathcal{N}^{(i)} \cap \mathcal{A}} A_{i,i'}\left(w^{(i,t)} - w^{(i',t)}\right)}_{\text{Byzantine neighbors}} \Bigg)
\end{aligned}
$$

Attack at Time t



$w^{(i',t)}$ received by node i from a

- ● honest neighbor $i' \in \mathcal{N}^{(i)} \setminus \mathcal{A}$
- ▲ Byzantine neighbor $i' \in \mathcal{N}^{(i)} \cap \mathcal{A}$

Two Strategies Against Byzantine Nodes

1. **Filter suspicious neighbors** update $w^{(i',t)}$
2. **Robust aggregation.** Instead of using the sum the neighbors updates, use a robust aggregator such as median or trimmed mean.

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

Basic Idea of Filtering

identify and ignore updates from suspicious neighbors.

- we can use anomaly or outlier detection methods.
- leverage own local dataset $\mathcal{D}^{(i)}$ for the filtering, e.g., by comparing received updates with gradient $\nabla L_i \left(w^{(i,t)}\right)$
- BUT: your own local dataset may be corrupted too (how much do you trust your smartphone?)

Filtering Methods

- **Krum.** Blanchard et al. (NeurIPS 2017), DOI: `10.5555/3294771.3294783`
  Selects the client update whose neighboring updates are closest in Euclidean distance. Idea: honest updates cluster around the true gradient, while Byzantine updates appear as geometric outliers.
- **FLTrust.** Cao et al. (NDSS 2021), DOI: `10.14722/ndss.2021.24434`
  Uses a small trusted *server dataset* to compute a reference gradient. Client updates are scored by cosine similarity to this reference and reweighted.

FLTrust

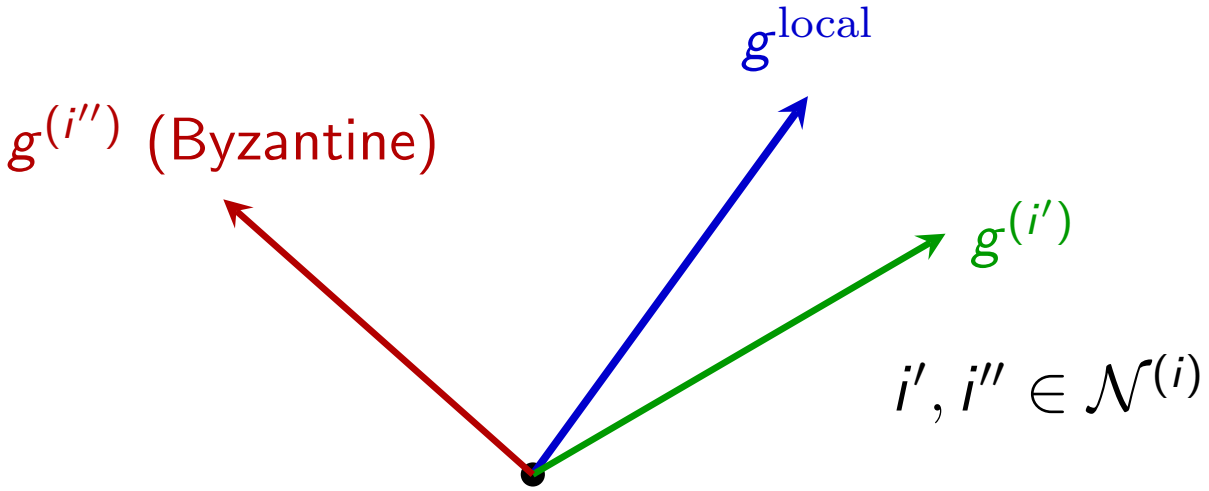
Local loss gradient (reference):

$$g^{\text{local}} = \nabla L_i \left(w^{(i,t)}\right)$$

Gradient component from neighbour i':

$$g^{(i')} = \alpha A_{i,i'} \left(w^{(i,t)} - w^{(i',t)}\right)$$

Filter neighbours by cosine similarity: $\mathrm{ReLU}\big(\cos\langle g^{(i')}, g^{\text{local}}\rangle\big)$



X. Cao et al., *FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping*, NDSS 2021.

Krum

Select update with min. distances to nearest neighbours

$$\hat{w}^{(\mathrm{Krum})} = w^{(i^*)}, \qquad i^* = \arg\min_{i' \in \mathcal{N}^{(i)}} \sum_{i'' \in \mathcal{K}^{(i')}} \left(w^{(i',t)} - w^{(i'',t)} \right)^2$$

here, $\mathcal{K}^{(i')}$ is the set of k nearest neighbours of $i' \in \mathcal{N}^{(i)}$

- **requirement:** $k > (|\mathcal{N}^{(i)}| - 2)/2$
- can tolerate up to $|\mathcal{N}^{(i)}| - k - 2$ Byzantine neighbors
- extension **Multi-Krum**

Krum – Illustration

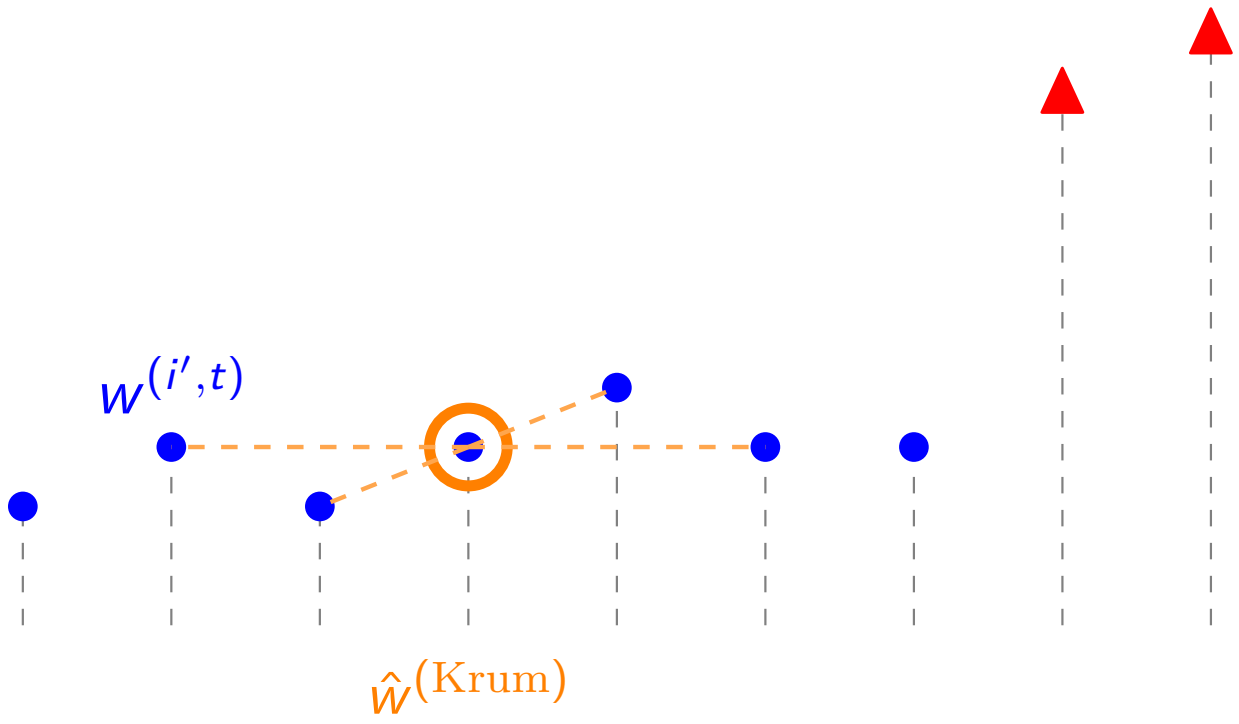


Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

From Gradient Step to Aggregation

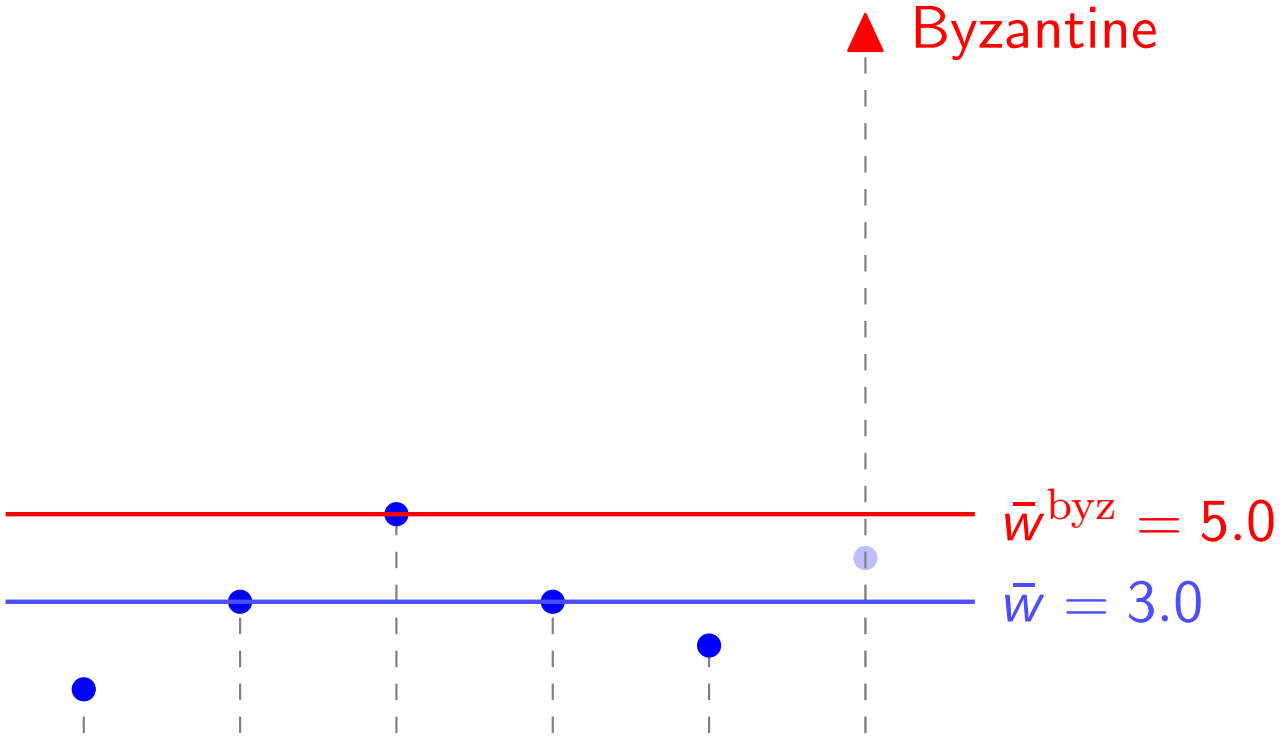
assume uniform edge weights $A_{i,i'} = 1$

gradient step at round t becomes

$$w^{(i,t+1)} = w^{(i,t)} - \eta\Bigg(\underbrace{\nabla L_i\left(w^{(i,t)}\right)}_{\text{local gradient}} + \underbrace{2\alpha\,|\mathcal{N}^{(i)}|\cdot w^{(i,t)}}_{\text{scaled current params}} - \underbrace{2\alpha\sum_{i'\in\mathcal{N}^{(i)}} w^{(i',t)}}_{\text{sum of neighbour params}}\Bigg)$$

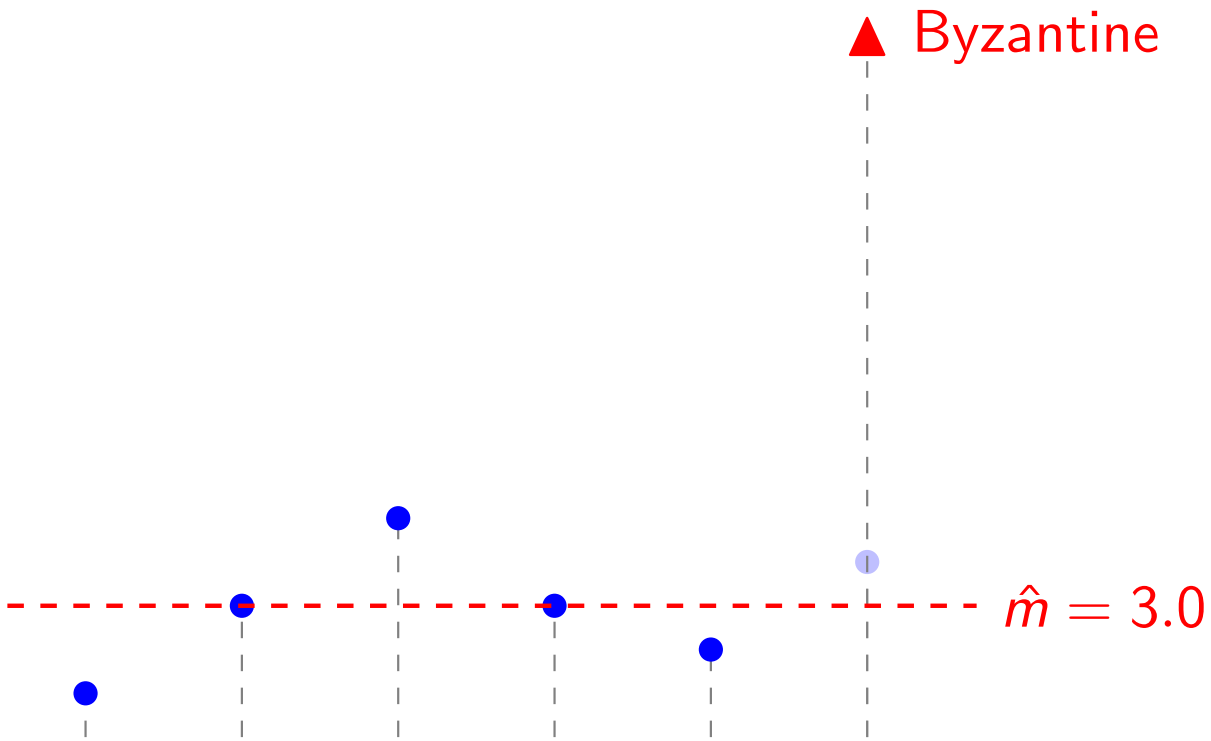
Aggregation problem: Byzantine $i' \in \mathcal{A}$ send *arbitrary* $w^{(i',t)}$ — how to aggregate $\left\{w^{(i',t)}\right\}_{i'\in\mathcal{N}^{(i)}}$ robustly?

Mean is Not Robust



one Byzantine node can perturb the mean arbitrarily;

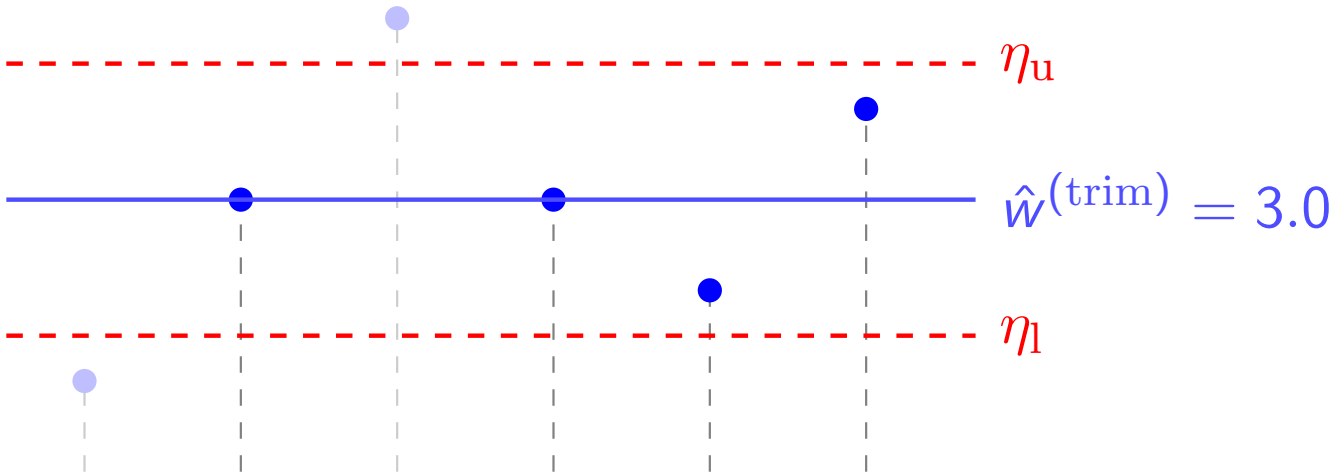
Median



Median $\hat{m} \in \arg\min_{w} \sum_{i' \in \mathcal{N}^{(i)}} \left| w^{(i',t)} - w \right|$

Dong Y,et.al.,*Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates* PMLR 80:5650-5659, 2018.

Trimmed Mean



$$\hat{w}^{(\mathrm{trim})} := \frac{1}{|\mathcal{N}^{(i)}|} \sum_{i' \in \mathcal{N}^{(i)}} \phi\left(w^{(i',t)}\right), \quad \phi(w) := \begin{cases} \eta_{\mathrm{u}} & w \geq \eta_{\mathrm{u}} \\ w & \eta_{\mathrm{l}} \leq w \leq \eta_{\mathrm{u}} \\ \eta_{\mathrm{l}} & w \leq \eta_{\mathrm{l}} \end{cases}$$

G. Lugosi and S. Mendelson, *Robust multivariate mean estimation: The optimality of trimmed mean*, 2021. DOI: 10.1214/20-AOS1961

From Gradient Step to Regularised Minimisation

gradient step at node i (uniform weights, $A_{i,i'} = 1$):

$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta \underbrace{\left(\nabla L_i\left(\mathbf{w}^{(i,t)}\right) + 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} \left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)}\right)\right)}_{\mathbf{g}^{(i,t)}}$$

This is **equivalent** to the regularised minimisation:

$$\mathbf{w}^{(i,t+1)} = \arg\min_{\mathbf{w} \in \mathbb{R}^d} \Big[\underbrace{\left\langle \mathbf{g}^{(i,t)}, \mathbf{w} \right\rangle}_{\text{linearised objective}} + \underbrace{\frac{1}{2\eta} \left\| \mathbf{w} - \mathbf{w}^{(i,t)} \right\|_2^2}_{\text{proximal term}} \Big]$$

Byzantine $i' \in \mathcal{A}$ corrupt $\mathbf{w}^{(i',t)} \Rightarrow$ corrupts $\mathbf{g}^{(i,t)}$

Robust Aggregation via Clipping (Online Setting)

replace gradient step with **clipped gradient** + **Huber regularizer**:

$$\mathbf{w}^{(i,t+1)} = \arg\min_{\mathbf{w}\in\mathbb{R}^d} \Big[\underbrace{\big\langle \widetilde{\mathbf{g}}^{(t,c)}, \mathbf{w} \big\rangle}_{\text{clipped gradient}} + \underbrace{R_t\big(\mathbf{w} - \mathbf{w}^{(i,t)}\big)}_{\text{Huber regularizer}} \Big]$$

where

$$\widetilde{\mathbf{g}}^{(t,c)} = \frac{\widetilde{\mathbf{g}}^{(t)}}{\|\widetilde{\mathbf{g}}^{(t)}\|} \min\big(h_t, \|\widetilde{\mathbf{g}}^{(t)}\|\big) \qquad \text{(clip to } h_t\text{)}$$

$$R_t(\mathbf{d}) = \begin{cases} \dfrac{\|\mathbf{d}\|^2}{2\eta} & \|\mathbf{d}\| \le h_t \\ h_t \|\mathbf{d}\| - \dfrac{h_t^2}{2\eta} & \|\mathbf{d}\| > h_t \end{cases} \qquad \text{(Huber: quadratic / linear)}$$

J. Zhang and A. Cutkosky, *Unconstrained Robust Online Convex Optimization*, arXiv:2506.12781, 2025.

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

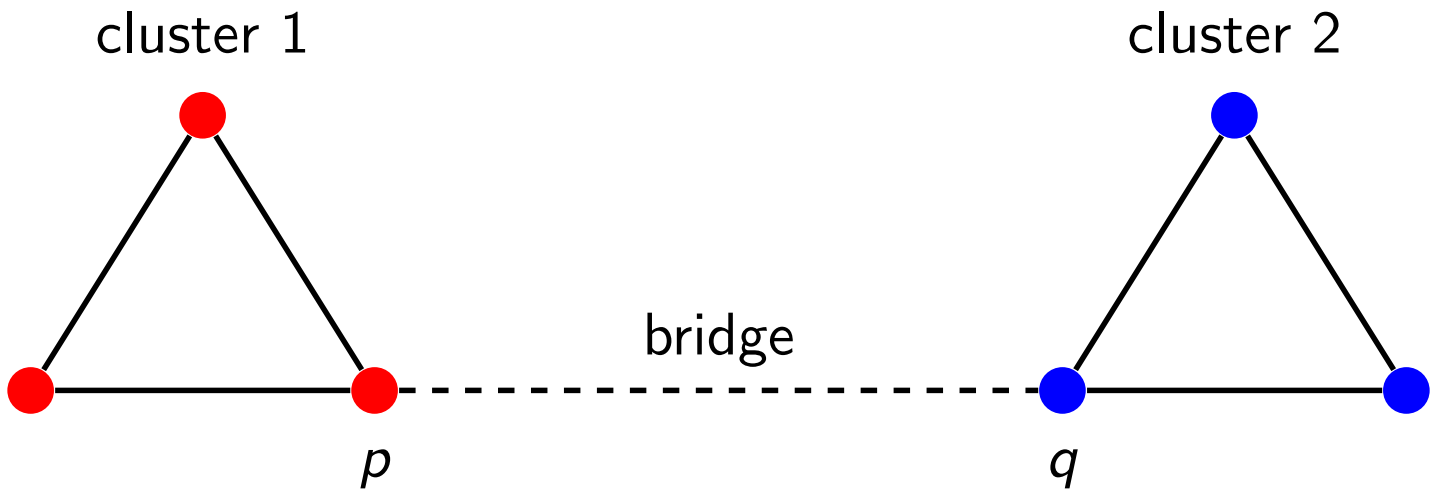
Summary

From Cyber Security to Statistical Robustness

So far we focused on **cyber security**: effects of *malicious* actors (Byzantine devices) that deliberately corrupt their updates.

But FL must also be robust against **statistical heterogeneity**: even *honest* nodes may have non-i.i.d. local data distributions, causing their updates to diverge.

Running Example: Two Clusters



simple probabilistic model for non-i.i.d. data:

- local dataset at node i consists of single value $y^{(i)}$
- cluster 1 (red): $y^{(i)} = 3 + \mathrm{noise}^{(i)}$,
- cluster 2 (blue): $y^{(i)} = -3 + \mathrm{noise}^{(i)}$

GTVMin and Closed-Form Solution

Applying (GTVMin-$\mathbb{R}$) to the running example:

$$\min_{\mathbf{w}\in\mathbb{R}^n} \sum_{i=1}^{n} \left(y^{(i)} - w^{(i)}\right)^2 + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left(w^{(i)} - w^{(i')}\right)^2$$

In matrix form, zero-gradient condition gives:

$$\left(\mathbf{I} + \alpha\, \mathbf{L}\right) \widehat{\mathbf{w}} \;=\; \mathbf{y}$$

Since $\mathbf{I} + \alpha\, \mathbf{L} \succ 0$, the unique closed-form solution is:

$$\boxed{\widehat{\mathbf{w}} \;=\; \left(\mathbf{I} + \alpha\, \mathbf{L}\right)^{-1} \mathbf{y}}$$

here $\mathbf{y} = \left(y^{(1)}, \ldots, y^{(n)}\right)^{\top}$ and $\mathbf{L}$ is the Laplacian matrix of the FL network.

Decoupled Solution: $A_{p,q} = 0$

If bridge weight $A_{p,q} = 0$, then $\mathbf{L}^{(\mathrm{inter})} = \mathbf{0}$ and $\mathbf{L} = \mathbf{L}^{(\mathrm{intra})}$ is block-diagonal. The solution decouples:

$$\widehat{\mathbf{w}} \;=\; \mathbf{P}^{(\mathrm{intra})}\,\mathbf{y} \;=\; \begin{pmatrix} \mathbf{P}^{(1)}\,\mathbf{y}^{(1)} \\ \mathbf{P}^{(2)}\,\mathbf{y}^{(2)} \end{pmatrix}$$

Each cluster is solved independently:

- cluster 1 (red): $\widehat{w}^{(i)} \approx 3$ for all $i \in \mathcal{C}^{(1)}$
- cluster 2 (blue): $\widehat{w}^{(i)} \approx -3$ for all $i \in \mathcal{C}^{(2)}$

Nodes in same cluster average their local values, unaffected by the other cluster.

Effect of Bridge: $A_{p,q} > 0$

Split the Laplacian as $\mathbf{L} = \mathbf{L}^{(\mathrm{intra})} + \mathbf{L}^{(\mathrm{inter})}$ (intra-cluster + bridge). Then using $(\mathbf{M}+\mathbf{N})^{-1} = \mathbf{M}^{-1} - \mathbf{M}^{-1}\mathbf{N}(\mathbf{M}+\mathbf{N})^{-1}$:

$$\widehat{\mathbf{w}} \;=\; \underbrace{\mathbf{P}^{(\mathrm{intra})}\,\mathbf{y}}_{\text{cluster-wise solution}} \;-\; \underbrace{\alpha\,\mathbf{P}^{(\mathrm{intra})}\,\mathbf{L}^{(\mathrm{inter})}\,\widehat{\mathbf{w}}}_{\text{bridge error}}$$

with bloack-diagonal $\mathbf{P}^{(\mathrm{intra})} = (\mathbf{I} + \alpha\mathbf{L}^{(\mathrm{intra})})^{-1}$ and

$$\mathbf{L}^{(\mathrm{inter})} \;=\; A_{p,q}\,(\mathbf{e}_p - \mathbf{e}_q)(\mathbf{e}_p - \mathbf{e}_q)^\top$$

- $A_{p,q} = 0$: clusters fully decoupled, $\widehat{\mathbf{w}} = \mathbf{P}^{(\mathrm{intra})}\,\mathbf{y}$
- $A_{p,q} > 0$: bridge pulls cluster solutions toward each other, error $\propto A_{p,q}$

Closed-Form via Sherman–Morrison

Since $\mathbf{L}^{(\mathrm{inter})} = A_{p,q}\,\mathbf{s}\mathbf{s}^\top$ is **rank-1** (with $\mathbf{s} := \mathbf{e}_p - \mathbf{e}_q$), the **Sherman–Morrison formula**

$$(\mathbf{M} + \mathbf{u}\mathbf{v}^\top)^{-1} \;=\; \mathbf{M}^{-1} - \frac{\mathbf{M}^{-1}\mathbf{u}\,\mathbf{v}^\top\mathbf{M}^{-1}}{1 + \mathbf{v}^\top\mathbf{M}^{-1}\mathbf{u}}$$

applied with $\mathbf{M} = \mathbf{I} + \alpha\mathbf{L}^{(\mathrm{intra})}$ and $\mathbf{u} = \mathbf{v} = \sqrt{\alpha A_{p,q}}\,\mathbf{s}$ gives:

$$\boxed{\widehat{\mathbf{w}} \;=\; \mathbf{P}^{(\mathrm{intra})}\,\mathbf{y} \;-\; \frac{\alpha A_{p,q}\;\mathbf{P}^{(\mathrm{intra})}\mathbf{s}\,\mathbf{s}^\top\mathbf{P}^{(\mathrm{intra})}\,\mathbf{y}}{1 + \alpha A_{p,q}\;\mathbf{s}^\top\mathbf{P}^{(\mathrm{intra})}\mathbf{s}}}$$

W. J. Sherman and J. Morrison, "Adjustment of an inverse matrix corresponding to a change in one element of a given matrix," *Ann. Math. Statist.*, vol. 21, no. 1, pp. 124–127, 1950.

Table of Contents

Motivation: Robustness and Security in FL

Running Example: Scalar FL with Clustered Model Parameters

Perturbed Gradient Descent

Filtering Suspicious Neighbors

Robust Aggregation Rules

Robustness Against Non-i.i.d. Data

Summary

Summary

- **Attack surface.** updates received from neighbors can be manipulated by Byzantine nodes;
- **Byzantine attacks.** backdoor, denial-of-service attack;
- **Filtering.** Krum (nearest-neighbour distance); FLTrust (similarity with local gradient);
- **Robust aggregation.** median and trimmed mean instead of mean.
- **Non-i.i.d. robustness.** GTVMin solution decomposes into cluster-wise part + bridge error $\propto A_{p,q}$

European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Official Journal of the European Union*, L series, Jul. 12, 2024. Available: https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng